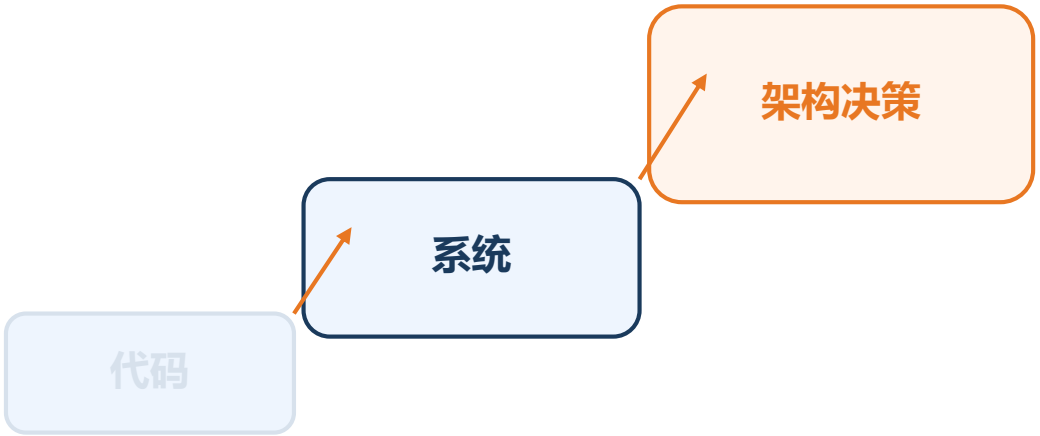
从“能写代码”到“能控复杂系统”

写代码解决局部问题，架构控制系统复杂性

软件体系结构 · 第1次课 · 2026年



问题冲击

同样会写代码，差距在哪里？

差距不在代码量，在复杂系统的控制力。

单人小项目

- 1人负责全部代码
- 1000行代码
- 流程简单、依赖少
- 能跑即可

能跑

多人复杂系统

- 5人以上协作
- 同样1000行代码
- 依赖、数据、发布互相影响
- 局部正确也可能整体失控

失控

工具箱

今天拿走三个工具

今天要拿走的是工具，不是概念。

1 复杂性四来源

识别“复杂在哪”

需求变化、协作规模、技术组合、运行约束

2 FR / QA / C 三分法

分析“需求是什么”

功能、质量属性、约束条件

3 架构时间账

判断“决策值不值”

提前投入与未来代价的比较

考试会给场景，要求用这三个工具分析

背景

软件工程历史演化

体系结构成为核心问题，是因为复杂性超过了个人经验上限。

1960s	1970s	1990s	2010s+
手工艺	**结构化 / 瀑布**	**OO / 敏捷**	**云原生 / 分布式**
代码=个人直觉	过程控制	迭代协作	复杂性爆炸

问题升级 → 方法升级；复杂性超过个人经验上限 → 架构成为核心问题

AI时代

AI 时代的判断边界

AI 能生成代码，但不能替你承担决策后果。

AI 能加速的工作

- 生成代码框架
- 补全样板代码
- 生成测试草稿
- 查 API / 改格式

实现变快

判断更贵

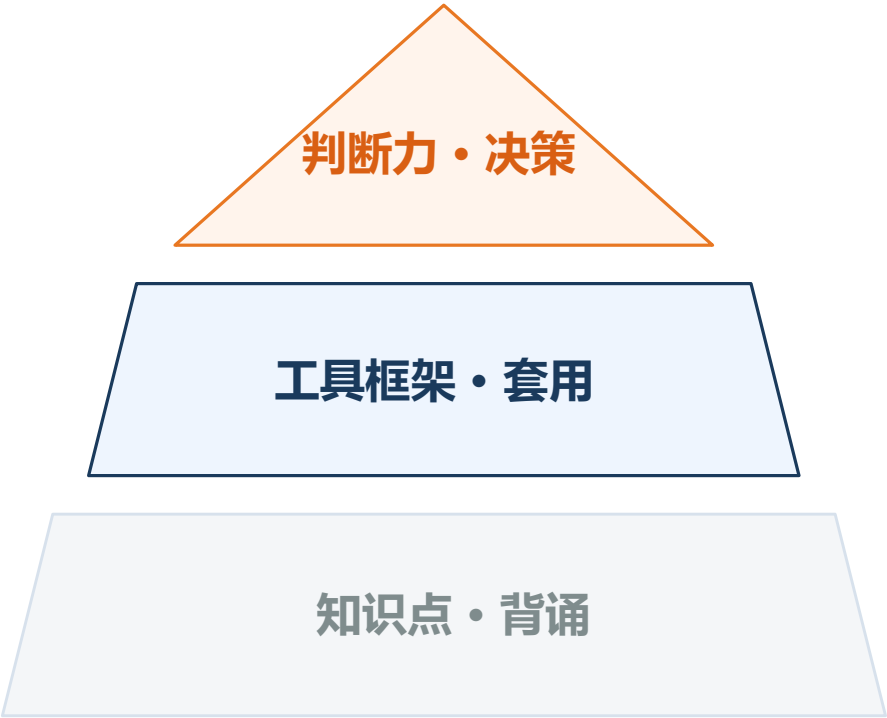
必须你来判断

- 质量属性权衡
- 约束条件识别
- 演进成本判断
- 上线风险责任

方法

判断力三层模型

本课不是背概念，是训练工程判断力。



教科书式

- 记住模型名称
- 背定义
- 看图识别

本课方式

- 给场景
- 识别约束
- 判断方案会怎样失败

考试考的是判断力，不是名词默写

案例

贯穿案例档案：实验室预约与设备管理系统

一个真实系统，全课持续分析。

字段	内容
目标用户	在校研究生、实验室管理员、设备负责人
业务高峰	考试周预约开放，10分钟内300人同时提交
数据对象	用户、设备、预约记录、冲突检测、审批流
部署环境	学校机房，不能随意使用公有云，无法频繁发布
初步风险	并发冲突、权限越权、审批流程变化

后续所有需求、架构、测试和项目训练都围绕这个系统展开

框架

复杂性四来源总览

复杂性不是“功能多”，而是四类来源共同叠加。

来源	失败数字	架构关注点
需求变化速率	24次变更，每次平均3天返工	可变性封装
协作规模	5人到10人，沟通路径10到45	模块边界与接口
技术组合深度	5层99%可用性降到95.1%	集成点管理
运行约束	300并发崩溃，补救3天	质量属性前置

复杂系统 = 需求变化速率 × 协作规模 × 技术组合深度 × 运行约束

复杂性 1/4

来源一：需求变化速率

需求变化危险，是因为边界、流程和角色会同时移动。

边界变

按学院管 → 按校区管

流程变

审批1级 → 审批3级

角色变

设备负责人1个 → 多个

你的系统里，哪类变化最快?

工程案例

需求变化失败案例：一个字段的代价

一个字段的决策，可能种下 9 个月后 2 个月返工。

```
department_name   VARCHAR(50)
```

Day 1

省30分钟
直接写死学院名

→

6个月后

学院合并
数据不一致，修1周

→

重构2个月

架构根因

- 把“学院”写成固定字段
- 没有抽象组织单元
- 权限边界变化时全系统受影响

当初省30分钟，后来付出2个月

方法

可变性封装方法

变化快的地方，不要让全系统都知道。

错误设计：硬编码学院名

- department_name VARCHAR(50)
- 全系统耦合“学院”概念
- 组织变化时多处代码和数据受影响

改进设计：抽象组织单元

- org_unit_id FK → org_units
- 组织变化只改一处
- 权限、流程、校区均可复用同一抽象

组织边界 / 权限边界 / 流程边界，任一可能变化 → 抽象

复杂性 2/4

来源二：协作规模

协作复杂性来自沟通路径的爆炸增长。

沟通路径 = n(n-1) / 2

1人
0条

5人
10条

10人
45条

20人
190条

人数增长带来的不是线性成本，而是沟通结构的改变

工程案例

Brooks 定律案例

人月不是线性的，新人加入可能让项目更慢。

场景

- 项目落后2周
- 再加入2名新人
- 希望赶上截止日期

真实代价

- 新人培训：2周
- 旧成员被打断：沟通路径增加
- 代码审查负担上升
- 项目可能再延迟3周

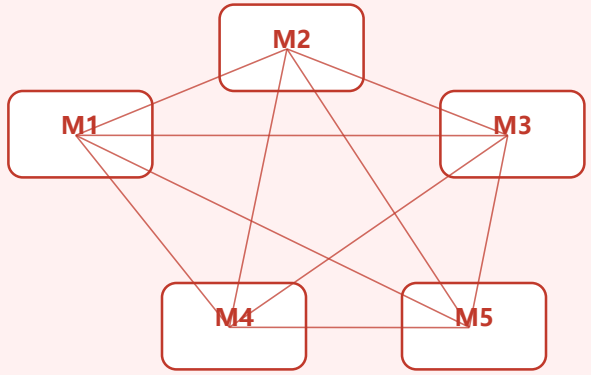
加人的代价，比你以为的贵得多

方法

模块化应对协作复杂性

好的模块边界不是画方框，是减少跨组沟通。

错误：全连接协作



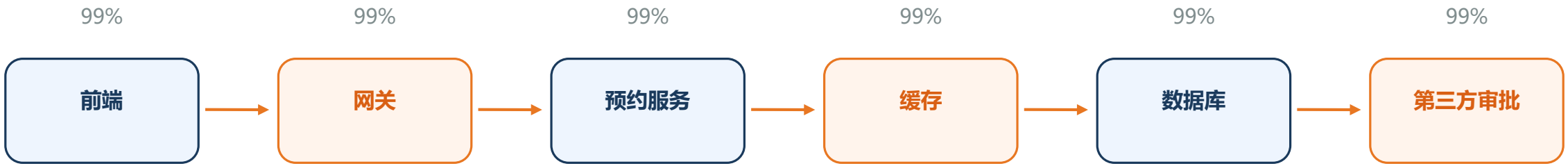
好的：稳定接口隔离

接口稳定 = 一组改动不需要另一组开会

复杂性 3/4

来源三：技术组合深度

技术栈越长，失败模式越多，可用性越低。



$$0.99^6 = 94.1\%$$

每加一层，整体可用性就下降

方法

可靠性衰减计算

0.99 连乘 5 次不是 0.99，而是 0.951。

依赖层数	整体可用性
1层	0.990
2层	0.980
3层	0.970
4层	0.961
5层	0.951

工程影响

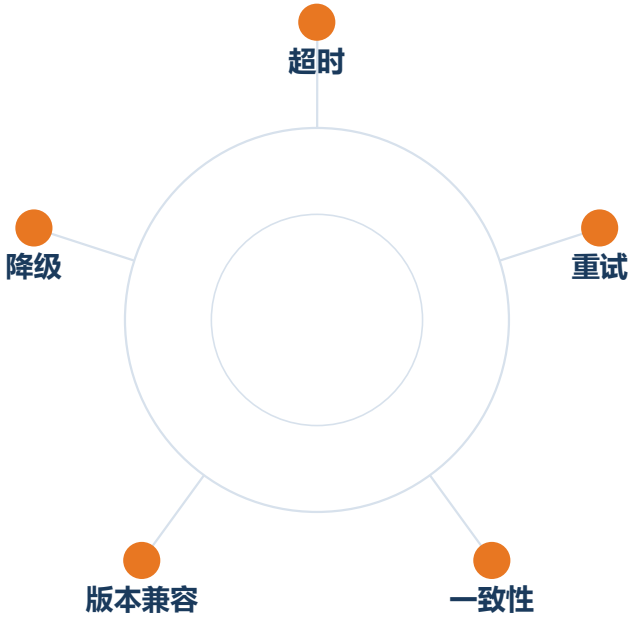
- 5层组件各自99%可用
- 整体约95.1%可用
- 一年停机约43小时

依赖层数是可用性设计的关键决策

方法

集成点风险雷达

每个集成点都是一个潜在失效模式。



给每个集成点贴风险标签

- 超时：调用方等多久？
- 重试：重试是否幂等？
- 一致性：谁是主数据？
- 版本兼容：能否同时部署？
- 降级：挂了还能运行吗？

接口能调通只是开始，失效模式设计才是架构工作

复杂性 4/4

来源四：运行约束

运行约束是系统上线后的真实世界，不是功能列表里的条目。

并发	同时多少人?
延迟	用户等多久?
可用性	允许宕机多久?
数据安全	谁能看什么?
部署限制	在哪里跑，能不能公有云?

这些不在功能列表里，但决定架构选择

工程案例

并发失效真实场景

300 人同时提交，系统崩溃，代价不只是宕机。

考试周 9:00 • 预约窗口开放 • 10分钟内300人同时提交

请求涌入 → 连接池耗尽 → 超时错误 → 用户重试 → 彻底宕机

代价

- 宕机3小时
- 补救3天
- 用户投诉53条

架构根因

- 没有限流
- 没有排队
- 数据库未按瓶颈设计

限流、排队、缓存、异步分别解决崩溃链路中的不同问题

方法

质量属性场景卡

质量属性必须可度量，“高性能”不算质量属性。

字段	实验室预约系统示例
场景	考试周300人预约
刺激	10分钟内请求涌入
环境	正常工作日 vs 考试周高峰
响应	页面响应 P95 < 2 秒
度量 / 降级	超出后排队提示，不返回 500

“高性能”不是质量属性；P95 < 2s 才是可验证约束

课堂训练

四来源快速诊断训练

识别复杂性，是架构工作的第一步。

来源	判断依据	学生评分
需求变化	审批流程和角色未来可能变化	高 / 中 / 低
协作规模	3-5人团队，跨组较少	高 / 中 / 低
技术组合	Web + DB + 通知 + 身份认证	高 / 中 / 低
运行约束	考试周300并发，校内部署	高 / 中 / 低

2分钟投票：哪一项是实验室预约系统最大的复杂性来源?

三分法

三分法引入：功能做完仍可能失败

功能只是需求分析的三分之一。

只有功能清单的需求说明书

- 学生预约
- 管理员审批
- 用户查询

缺失的问题

- 考试周能扛住吗？
- 数据能不能丢？
- 能不能上公有云？
- 谁能访问学生数据？

功能做完了，只代表 FR 可能完成；QA 和 C 仍可能缺失

三分法

FR / QA / C 三分法总览

FR 问做什么，QA 问做到什么程度，C 问不能违反什么。

FR 功能需求

- 问：系统做什么？
- 预约设备
- 审批预约
- 冲突检测

QA 质量属性

- 问：做到什么程度？
- P95 < 2s
- 99.9%可用
- 越权0容忍

C 约束条件

- 问：不能违反什么？
- 学校机房
- 不能公有云
- 数据不出网

名词解释和综合案例都会考三分法辨析

三分法

功能需求定义与范围

功能需求不等于页面按钮，系统行为都算。

误解：页面按钮

- 登录
- 预约
- 查询

正确：系统行为

- 核心业务：预约设备
- 流程控制：审批预约
- 业务规则：冲突检测
- 异步触发：通知提醒

系统需要做什么响应、计算、存储、通知，都是 FR

三分法

质量属性专业写法

质量属性要有指标、环境和阈值。

质量属性	可度量写法
性能	P95响应时间 < 2秒（正常负载下）
可用性	99.9%（年停机 < 8.7小时）
安全性	未授权访问0容忍；JWT过期15分钟
可维护性	小功能从提出到上线 < 3天

写“高性能”是0分；写“P95 < 2s”才算质量属性

三分法

约束条件三类型

约束条件决定什么方案不可选，不是说明什么方案更优。

技术约束	Python 3.8+，MySQL，机房无 Redis 许可
组织约束	3人团队，10周工期，导师期末看
法规/安全约束	学生数据不出校园网，审计日志保存1年

只要违反约束，方案再优也不可选

方法

三分法检查清单

三分法不是分类，是检查遗漏的三把尺子。

FR 检查

- ☐ 有没有遗漏使用场景？用户最重要的动作都覆盖了吗？

QA 检查

- ☐ 每条 QA 可度量吗？有没有忽视性能、安全、可用性？

C 检查

- ☐ 有没有“以为可以但其实不行”的假设？

M1 需求文档：先用这三把尺子自查

工程案例

贯穿案例三分法完整示例

三分法用在实验室预约系统上，应当形成可检查表。

FR 功能需求	QA 质量属性	C 约束条件
预约设备	P95 < 2s	学校机房部署
审批预约	99.9%可用	数据不出校园网
冲突检测	越权访问0容忍	审计日志保存1年
通知提醒	故障后30分钟内恢复	3人团队10周内完成

学生项目也要提交同样结构：FR / QA / C

课堂训练

F / Q / C 快速判断训练

判断 FR/QA/C，关键看动词、度量词和限制词。

1. 系统需要支持微信扫码登录	**F**	Q	C
2. 数据不能丢失	F	**Q**	C
3. 必须部署在学校机房，不能使用公有云	F	Q	**C**
4. 用户可以查看过去30天的预约历史	**F**	Q	C

判断线索

动词通常指向 FR；度量词通常指向 QA；限制词通常指向 C。

答案：1-F / 2-Q / 3-C / 4-F

方法

方法迁移地图

三分法以后每个阶段都会用。

需求
写 FR/QA/C 表

→

架构
验证 QA 与约束

→

测试
设计验证用例

→

答辩
解释取舍与风险

后续文档里必须能看到 FR / QA / C

架构观

常见误解澄清

架构不是阶段，是贯穿全程的关键决策集合。

~~体系结构 = 设计阶段画图~~

体系结构 = 贯穿全程的关键决策集合

完整架构表达至少包含

- 决策理由
- 质量属性
- 约束条件

只有框和箭头，没有决策理由、QA、约束，不算完整架构

架构观

架构决策层贯穿生命周期

架构决策不是一个节点，而是贯穿全流程的决策层。

架构决策层：质量属性、模块边界、部署形态、演进策略

- 需求
- 设计
- 实现
- 测试
- 发布运维
- 演进

架构影响测试：测试要验证架构假设，而不只是点功能

架构观

各阶段架构要回答的问题

每个阶段，架构都要回答一个核心问题。

阶段	架构要回答的问题
需求	哪些 QA 最关键?
设计	哪种结构满足这些 QA?
实现	模块边界够稳定吗?
测试	如何验证架构假设?
发布	能安全部署和回滚吗?
演进	技术债在哪里，代价多高?

给某个阶段，就要能说出架构关注的问题

工程案例

时间账案例：字段方案对比

一行代码，种下了 9 个月后 2 个月返工的种子。

```
department_name  VARCHAR(50)
```

方案	字段设计	长期代价
省事版	department_name VARCHAR(50)	重构2个月
中间版	department_id FK → dept	中等代价
稳妥版	org_unit_id FK → org_units	最低代价

比较方案时，不只看今天省事，还要看未来变化由谁承受

工程案例

架构时间账

当初省 30 分钟，后来付出 2 个月。

30分钟 vs 2个月

Day 1

省30分钟
直接写死

6个月后

修1周
学院合并

9个月后

重构2个月
权限爆发

决策成本在早期，代价在后期

方法

架构决策卡

好的架构思维，是在早期问对四个问题。

变化概率

这里将来有多大可能变化？
高 / 中 / 低

修改代价

如果变了，改起来多贵？
估算人天

提前投入

现在多做一步，要多花多少？
估算人天

决策结论

变化概率 × 修改代价 > 提前投入
→ 抽象值得

综合题可要求填写简版 ADR 或架构决策卡

课堂训练

课堂演练：你的项目复杂在哪里?

先写 5 分钟，再说，不举手也没关系。

任务

- 你做过最复杂的项目是什么？（一句话）
- 它最复杂的地方属于哪类复杂性?
- 这个复杂性影响了系统结构吗？怎么影响的?

示例

宿舍报修系统：高峰并发 + 派单规则变化 → 任务队列 + 规则引擎

先独立写5分钟；愿意分享就说，不分享也没关系

收束

两把尺子

今天拿走两把尺子，后续每个项目都能用。

复杂性判断尺

- 用途：识别项目复杂在哪
- 工具：四来源 × 高/中/低
- 结果：定位架构风险

需求分析尺

- 用途：确保需求完整且可测
- 工具：FR / QA / C 三分法
- 结果：形成可检查表

说出两把尺子的使用场景，就是本节课的最低达成目标

过渡

下一讲悬念

同一个项目，为什么有的适合瀑布，有的必须迭代?

同一个项目，
为什么有的适合瀑布，有的必须迭代?

课前作业：完成选题卡，下次课开始时展示

作业

选题卡格式

选题卡是项目训练的起点。

1. 项目名称	实验室预约系统
2. 目标用户	研究生、管理员
3. 核心场景	预约 / 审批 / 通知
4. 最大复杂性	考试周并发高，流程会变化
5. 一个 QA	P95 < 2s
6. 一个约束	学生数据不出校园网

下次课没有选题卡 = 没法参与第一个活动

项目

课程项目产出

最终要交出的不是一个想法，而是一组可评审工程文档。

V1 定义

- 可被同伴指出问题的工作版本
- 不是完美稿
- 是可讨论稿

3个里程碑

- M1 需求
- M2 架构期中
- M3 答辩

最终产出

- 需求表
- 三分法表
- 架构图
- 风险说明
- 答辩

项目文档质量会直接影响平时成绩和答辩表现

考试锚点

本讲三条结论

这三条就是今天的考试重点。

1	复杂系统差距在复杂性识别和应对能力	考试：给案例识别复杂性来源
2	三分法：功能 / 质量属性 / 约束，QA 必须可度量	考试：判断 QA，写出可度量形式
3	架构是全程决策集合，成本在早期，代价在后期	考试：分析架构决策的时间账

复习时每条结论都要能对应一个案例

考试

考试说明

每次课结尾给考点锚定，三条结论就是复习重点。

考试形式

闭卷 · 100分钟
60%分数

题型三类

- 名词解释：关键词定义
- 简答题：给场景用框架分析
- 综合案例题：识别复杂性 + 三分法 + 架构风险

复习方法：每次课3条结论 + 1个案例 + 1个框架表

交流

问题交流

把问题带出来，比假装听懂更重要。

Questions?

哪个概念最难?

哪个案例最像你的项目?

你的选题可能复杂在哪里?

可以提交一个问题，作为课堂参与记录

结束

带着工具看项目

从今天开始，看项目先看复杂性、需求三分法和时间账。

复杂性四来源

先判断复杂在哪

FR / QA / C

再检查需求是否完整

架构时间账

最后比较决策代价

课前作业：带选题卡进入下一次课